

WINGMAN FRAMEWORK

QueryBuilder & QueryableModel

Developer Reference Documentation

Overview

Wingman's query layer is split into two classes that work together:

Class	Responsibility
<code>QueryBuilder</code>	Standalone SQL builder. Accepts a table name, collects clauses via fluent methods, and executes queries directly against a mysqli connection.
<code>QueryableModel</code>	Extends <code>BaseModel</code> . Wraps <code>QueryBuilder</code> and exposes all query methods directly on your model class, auto-resetting state after each execution.

Class Hierarchy

```
BaseModel      ← schema building (build, demolish, columns)
  ↑
QueryableModel ← query building (get, insert, update, delete...)
  ↑
UserModel      ← your actual model
```

Namespace

```
Wingman\Core\Database\QueryBuilder
Wingman\Core\Bases\QueryableModel
```

Setup

Extending QueryableModel

All your models should extend [QueryableModel](#) instead of [BaseModel](#):

```
<?php

namespace Wingman\App\Models;

use Wingman\Core\Bases\QueryableModel;

class UserModel extends QueryableModel
{
    protected string $table = 'users';

    public function schema(): void
    {
        $this->primaryKey()
            ->characters('username', 50)->unique()->required()
            ->characters('email', 255)->unique()->required()
            ->characters('password_hash', 255)->required()
            ->boolean('active', true);
    }
}
```

Instantiation

```
$user = new UserModel($db);    // $db is your mysqli connection
```

Reading Data

all()

Fetches every row from the table.

```
$users = $user->all();  
// SELECT * FROM users
```

get()

Executes the query with all applied clauses.

```
$users = $user->where('active', '=', 1)->get();  
// SELECT * FROM users WHERE active = 1
```

first()

Returns only the first matching row as an associative array, or `null` if not found.

```
$u = $user->where('email', '=', 'kyle@example.com')->first();  
// SELECT * FROM users WHERE email = ? LIMIT 1
```

find()

Shorthand for fetching a single row by primary key.

```
$u = $user->find(1);  
// SELECT * FROM users WHERE id = ? LIMIT 1  
  
// Custom primary key:  
$u = $user->find(42, 'uid');  
// SELECT * FROM users WHERE uid = ? LIMIT 1
```

count()

Returns the number of rows matching the applied conditions.

```
$total = $user->where('active', '=', 1)->count();  
// SELECT COUNT(*) as aggregate FROM users WHERE active = 1
```

select()

Specify which columns to retrieve.

```
$users = $user->select('id', 'username', 'email')->get();  
// SELECT id, username, email FROM users
```

Where Conditions

All where methods are chainable and use parameterized bindings to prevent SQL injection.

where() / orWhere()

```
// AND condition
$users = $user->where('active', '=', 1)
            ->where('role', '=', 'admin')
            ->get();
// WHERE active = 1 AND role = 'admin'

// OR condition
$users = $user->where('role', '=', 'admin')
            ->orWhere('role', '=', 'moderator')
            ->get();
// WHERE role = 'admin' OR role = 'moderator'
```

whereIn() / whereNotIn()

```
$users = $user->whereIn('role', ['admin', 'moderator', 'editor'])->get();
// WHERE role IN ('admin', 'moderator', 'editor')

$users = $user->whereNotIn('status', ['banned', 'suspended'])->get();
// WHERE status NOT IN ('banned', 'suspended')
```

whereNull() / whereNotNull()

```
$users = $user->whereNull('deleted_at')->get();
// WHERE deleted_at IS NULL

$users = $user->whereNotNull('email_verified_at')->get();
// WHERE email_verified_at IS NOT NULL
```

whereBetween()

```
$users = $user->whereBetween('age', 18, 35)->get();
// WHERE age BETWEEN 18 AND 35
```

whereLike()

```
$users = $user->whereLike('email', '%@gmail.com')->get();  
// WHERE email LIKE '%@gmail.com'
```

Joins

join() — INNER JOIN

```
$results = $post
    ->select('posts.id', 'posts.title', 'users.username')
    ->join('users', 'posts.user_id', '=', 'users.id')
    ->get();
// SELECT posts.id, posts.title, users.username
// FROM posts INNER JOIN users ON posts.user_id = users.id
```

leftJoin()

```
$results = $user
    ->select('users.username', 'posts.title')
    ->leftJoin('posts', 'users.id', '=', 'posts.user_id')
    ->get();
// FROM users LEFT JOIN posts ON users.id = posts.user_id
```

rightJoin()

```
$results = $post
    ->rightJoin('users', 'posts.user_id', '=', 'users.id')
    ->get();
// FROM posts RIGHT JOIN users ON posts.user_id = users.id
```

crossJoin()

```
$results = $category->crossJoin('products')->get();
// FROM categories CROSS JOIN products
```


Grouping & Ordering

groupBy() + having()

```
$results = $order
    ->select('user_id', 'COUNT(*) as total_orders')
    ->groupBy('user_id')
    ->having('total_orders', '>', 5)
    ->get();
// GROUP BY user_id HAVING total_orders > 5
```

orderBy() / orderByDesc()

```
$users = $user->orderBy('username')->get();
// ORDER BY username ASC

$users = $user->orderByDesc('created_at')->get();
// ORDER BY created_at DESC

// Multiple order columns:
$users = $user->orderBy('role')->orderByDesc('created_at')->get();
// ORDER BY role ASC, created_at DESC
```

Pagination

limit() + offset()

```
$users = $user->limit(10)->offset(20)->get();  
// SELECT * FROM users LIMIT 10 OFFSET 20
```

paginate()

Sets LIMIT and OFFSET from a page number automatically.

```
$users = $user->paginate(page: 3, perPage: 10)->get();  
// SELECT * FROM users LIMIT 10 OFFSET 20
```

paginateResult()

Returns results along with pagination metadata. Ideal for REST API responses.

```
$result = $user->where('active', '=', 1)->paginateResult(page: 2, perPage: 10);  
  
// Returns:  
// [  
//   'data'      => [...],  
//   'total'     => 143,  
//   'per_page'  => 10,  
//   'current_page' => 2,  
//   'last_page' => 15,  
//   'from'      => 11,  
//   'to'        => 20,  
// ]
```

Writing Data

insert()

Inserts a single row. Returns the new row's auto-increment ID.

```
$id = $user->insert([
    'username'    => 'kyle',
    'email'       => 'kyle@example.com',
    'password_hash' => password_hash('secret', PASSWORD_BCRYPT),
    'active'      => 1,
]);

echo $id; // e.g. 42
```

insertMany()

Inserts multiple rows in a single query. Returns the number of affected rows.

```
$count = $user->insertMany([
    ['username' => 'kyle', 'email' => 'kyle@example.com'],
    ['username' => 'john', 'email' => 'john@example.com'],
    ['username' => 'alice', 'email' => 'alice@example.com'],
]);

echo $count; // 3
```

update()

Updates rows matching the applied `where()` conditions. Returns the number of affected rows.

```
$affected = $user->where('id', '=', 1)->update([
    'username' => 'kyled',
    'active'   => 0,
]);

// UPDATE users SET username = ?, active = ? WHERE id = 1
```

delete()

Deletes rows matching the applied `where()` conditions. Returns the number of affected rows.

```
$affected = $user->where('id', '=', 1)->delete();  
// DELETE FROM users WHERE id = 1  
  
// Delete multiple:  
$affected = $user->whereIn('id', [3, 5, 8])->delete();  
// DELETE FROM users WHERE id IN (3, 5, 8)
```

Complex Query Examples

The following examples combine multiple clauses to demonstrate real-world usage.

Example 1 — Filtered, joined, paginated feed

Fetch active posts with author info, filtered by category, ordered by date, paginated.

```
$result = $post
->select('posts.id', 'posts.title', 'posts.created_at', 'users.username')
->join('users', 'posts.user_id', '=', 'users.id')
->where('posts.active', '=', 1)
->whereIn('posts.category', ['tech', 'design', 'php'])
->whereNotNull('posts.published_at')
->orderByDesc('posts.created_at')
->paginateResult(page: 1, perPage: 15);
```

Example 2 — Search with multiple OR conditions

Search users by username or email, exclude banned accounts.

```
$users = $user
->select('id', 'username', 'email', 'role')
->whereLike('username', '%kyle%')
->orWhere('email', 'LIKE', '%kyle%')
->whereNotIn('status', ['banned', 'suspended'])
->orderBy('username')
->limit(20)
->get();
```

Example 3 — Aggregation with groupBy and having

Find power users who have placed more than 10 orders in the last 30 days.

```
$powerUsers = $order
->select('user_id', 'COUNT(*) as order_count', 'SUM(total) as revenue')
->where('status', '=', 'completed')
->whereBetween('created_at', '2024-01-01', '2024-01-31')
->groupBy('user_id')
->having('order_count', '>', 10)
->orderByDesc('revenue')
->get();
```

Example 4 — Multi-join report

Build a report joining orders, users, and products.

```
$report = $order
->select(
    'orders.id',
    'users.username',
    'products.name as product_name',
    'orders.total'
)
->join('users', 'orders.user_id', '=', 'users.id')
->join('products', 'orders.product_id', '=', 'products.id')
->where('orders.status', '=', 'completed')
->whereNotNull('orders.shipped_at')
->orderByDesc('orders.created_at')
->limit(100)
->get();
```

Example 5 — Conditional update

Deactivate all unverified users who registered more than 7 days ago.

```
$affected = $user
->where('email_verified', '=', 0)
->where('created_at', '<', date('Y-m-d', strtotime('-7 days')))
->update(['active' => 0]);

echo "Deactivated {$affected} users.";
```

Method Reference

Read Methods

Method	Signature	Returns
all()	<code>all(): array</code>	array
get()	<code>get(): array</code>	array
first()	<code>first(): array null</code>	array null
find()	<code>find(int \$id, string \$pk = 'id'): array null</code>	array null
count()	<code>count(): int</code>	int
paginateResult()	<code>paginateResult(int \$page, int \$perPage): array</code>	array

Write Methods

Method	Signature	Returns
insert()	<code>insert(array \$data): int</code>	int (new ID)
insertMany()	<code>insertMany(array \$rows): int</code>	int (row count)
update()	<code>update(array \$data): int</code>	int (affected)
delete()	<code>delete(): int</code>	int (affected)

Clause Methods

Method	Signature	Returns
select()	<code>select(string ...\$columns): static</code>	static
where()	<code>where(col, op, val): static</code>	static
orWhere()	<code>orWhere(col, op, val): static</code>	static
whereIn()	<code>whereIn(col, array): static</code>	static
whereNotIn()	<code>whereNotIn(col, array): static</code>	static
whereNull()	<code>whereNull(col): static</code>	static
whereNotNull()	<code>whereNotNull(col): static</code>	static
whereBetween()	<code>whereBetween(col, min, max): static</code>	static
whereLike()	<code>whereLike(col, pattern): static</code>	static

join()	<code>join(table, first, op, second): static</code>	static
leftJoin()	<code>leftJoin(table, first, op, second): static</code>	static
rightJoin()	<code>rightJoin(table, first, op, second): static</code>	static
crossJoin()	<code>crossJoin(table): static</code>	static
groupBy()	<code>groupBy(string ...\$columns): static</code>	static
having()	<code>having(col, op, val): static</code>	static
orderBy()	<code>orderBy(col, dir = 'ASC'): static</code>	static
orderByDesc()	<code>orderByDesc(col): static</code>	static
limit()	<code>limit(int): static</code>	static
offset()	<code>offset(int): static</code>	static
paginate()	<code>paginate(page, perPage = 10): static</code>	static
toSQL()	<code>toSQL(): string</code>	string

Notes & Best Practices

Auto-reset

Query state is automatically reset after every execution method (`get()`, `first()`, `insert()`, etc.). You do not need to call `reset()` manually between queries on the same model instance.

SQL Injection Safety

All value bindings use parameterized `?` placeholders via `mysqli` prepared statements. Column names passed to `where()`, `orderBy()`, and join methods are interpolated directly — never pass user input as column names.

Debugging

Use `toSQL()` to inspect the generated SQL before executing:

```
echo $user->where('active', '=', 1)->orderByDesc('created_at')->toSQL();  
// SELECT * FROM users WHERE active = ? ORDER BY created_at DESC
```

enum() Fix

When defining enum columns in your model schema, values are automatically quoted:

```
$this->enum('role', ['admin', 'user', 'moderator'], 'user');  
// role ENUM('admin', 'user', 'moderator') DEFAULT 'user'
```